Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 13 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

Objective

The objective of Week 13 was to introduce configuration management for the Automated Grocery Inventory Dashboard using Ansible.

The configuration specification identifies the software prerequisites, application user, directories, configuration files, network ports and running services required by the project. Ansible was selected to automate the configuration and verification of the local target node.

The playbook was designed to be repeatable and idempotent. The first execution created the required configuration file, while the subsequent execution verified that no unnecessary changes were made.

Week 13 Deliverables

- Ansible inventory for the target node.
- Configuration management playbook.
- Application environment configuration file.
- First execution log.
- Verification of software prerequisites, ports and services.
- Idempotence validation.
- GitHub feature branch and pull request.
- Week 13 baseline tag `v0.13.0`.

1. 1. Configuration Specification

The local macOS system was used as the target node because the project development, Jenkins, MySQL and Docker environment is already hosted on the same machine.

1.1 1.1 Server Prerequisites

Category	Requirement	Purpose
Operating System	macOS	Target configuration node
Java	JDK 21	Spring Boot application runtime
Maven	3.9.16	Build and package the application
Git	2.55.0	Source-code version control
Docker	29.8.1	Container build and deployment
MySQL	26.7.0	Application database
Ansible	2.21.4	Configuration management
Python	3.14.7	Ansible execution environment

Table 1: Software prerequisites

1.2 1.2 Users, Folders and Files

The existing local application user is used to execute the project tooling. The playbook verifies that this user exists rather than creating an additional operating-system account.

The following directories and files are managed by the playbook:

```
automated-grocery-inventory-dashboard/
|
+-- ansible/
|   +-- inventory.ini
|   +-- configure.yml
|   +-- first-execution.log
|
+-- config/
    +-- application-environment.txt
```

The configuration directory is created with mode `0755`, while the environment configuration file is maintained with mode `0644`.

1.3 1.3 Network Ports and Services

Service	Port	Verification
MySQL	3306	Listening database service
Jenkins	8080	Jenkins server listening
Grocery Inventory App	8083	Docker-deployed application
Docker Registry	5001	Local Docker registry

Table 2: Required network services

The playbook also verifies Docker availability using `docker info` and checks the status of the two project-related containers: `grocery-inventory-cd` and `grocery-local-registry`.

2. 2. Ansible Inventory

The project uses a local inventory because the target node is the same macOS machine on which Ansible is executed.

```
[app]
localhost ansible_connection=local
```

The inventory was successfully tested using the Ansible ping module:

```
ansible -i inventory.ini app -m ping
```

The target returned:

```
localhost | SUCCESS
changed: false
ping: pong
```

3. 3. Ansible Configuration Playbook

The main playbook is stored at:

```
ansible/configure.yml
```

The playbook performs configuration and verification tasks in the following sequence:

1. Gather target-system information.
2. Verify the application user.
3. Ensure required project directories exist.
4. Create the application environment configuration file.
5. Verify Java, Maven, Git, Docker, MySQL and Ansible.
6. Verify required network ports.
7. Verify Docker service availability.
8. Check the application and registry containers.
9. Display the final configuration status.

The configuration file generated by the playbook records the environment requirements and service information:

```
Grocery Inventory Dashboard - Environment Configuration
==========================================================

Project Directory:
.../automated-grocery-inventory-dashboard

Application User: prachipatil

Java Requirement: JDK 21
Maven Requirement: 3.9.16
Git Requirement: 2.55.0
Docker Requirement: 29.8.1
MySQL Requirement: 26.7.0
Ansible Requirement: 2.21.4
```

```
MySQL Port: 3306
Jenkins Port: 8080
Application Port: 8083
Docker Registry Port: 5001
```

4. 4. First Ansible Execution

The playbook was executed using:

```
ansible-playbook -i inventory.ini configure.yml
```

The execution completed successfully.

The execution summary was:

```
localhost : ok=23
changed=1
unreachable=0
failed=0
skipped=0
rescued=0
ignored=0
```

The single change corresponded to creation/update of the application environment configuration file. All prerequisite, port and service checks completed successfully.

4.1 4.1 Verified Environment

The first execution confirmed:

- Java 21.0.12.1
- Maven 3.9.16
- Git 2.55.0
- Docker 29.8.1
- MySQL 26.7.0
- Ansible 2.21.4
- MySQL port 3306 listening
- Jenkins port 8080 listening
- Application port 8083 listening
- Registry port 5001 listening
- Docker service running
- Grocery Inventory container running
- Docker Registry container running

5. 5. Idempotence Verification

The same playbook was executed a second time without changing the target environment.

The second execution produced:

```
localhost : ok=23
changed=0
unreachable=0
failed=0
skipped=0
rescued=0
ignored=0
```

The `changed=0` result demonstrates that the playbook does not unnecessarily modify an already configured node.

6. 6. Version Control and GitHub Integration

The Week 13 configuration work was developed on a dedicated feature branch:

```
feature/ansible-configuration-management
```

The following files were committed:

- `ansible/configure.yml`
- `ansible/inventory.ini`
- `ansible/first-execution.log`
- `config/application-environment.txt`

The configuration was committed using:

```
git commit -m "Add Ansible configuration management"
```

Commit:

```
f56709d Add Ansible configuration management
```

A pull request was created from the feature branch into `develop`.

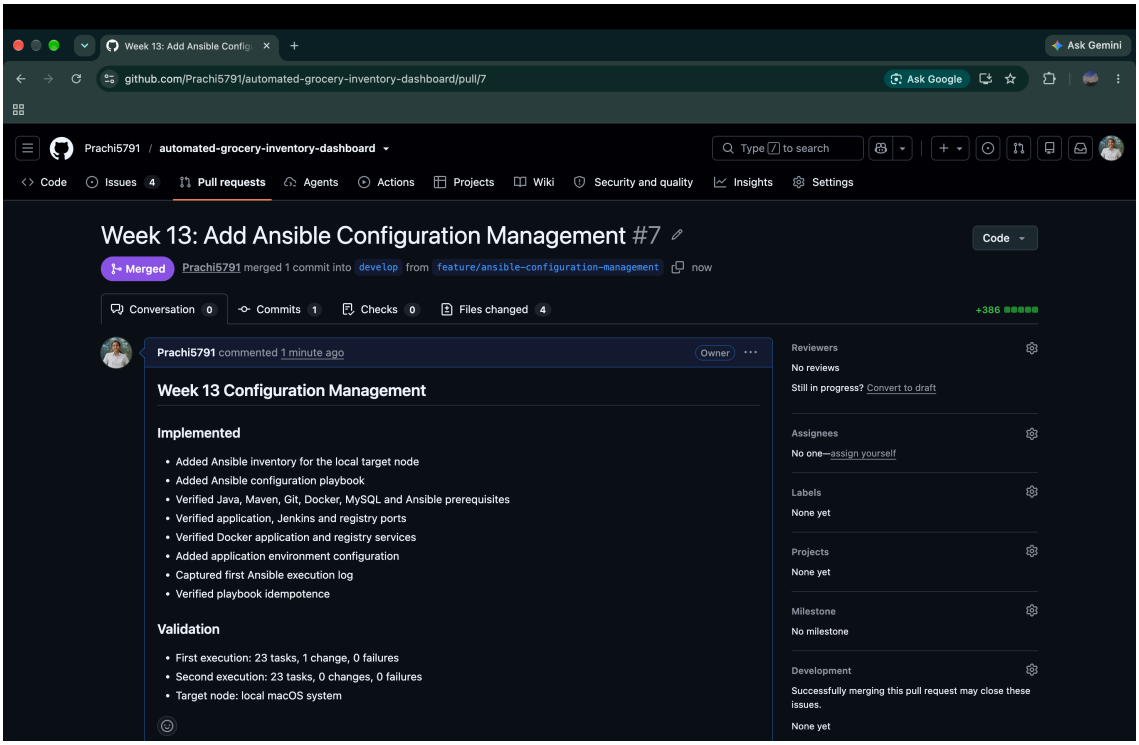


Figure 1: Week 13 GitHub pull request merged into develop

The pull request was successfully merged into `develop`. The local branch was then synchronized with the remote repository, and the working tree was confirmed clean.

7. 7. Baseline Tag

After merging the configuration management work, a versioned Week 13 baseline was created:

```
git tag -a v0.13.0 \
-m "Week 13: Ansible Configuration Management"

git push origin v0.13.0
```

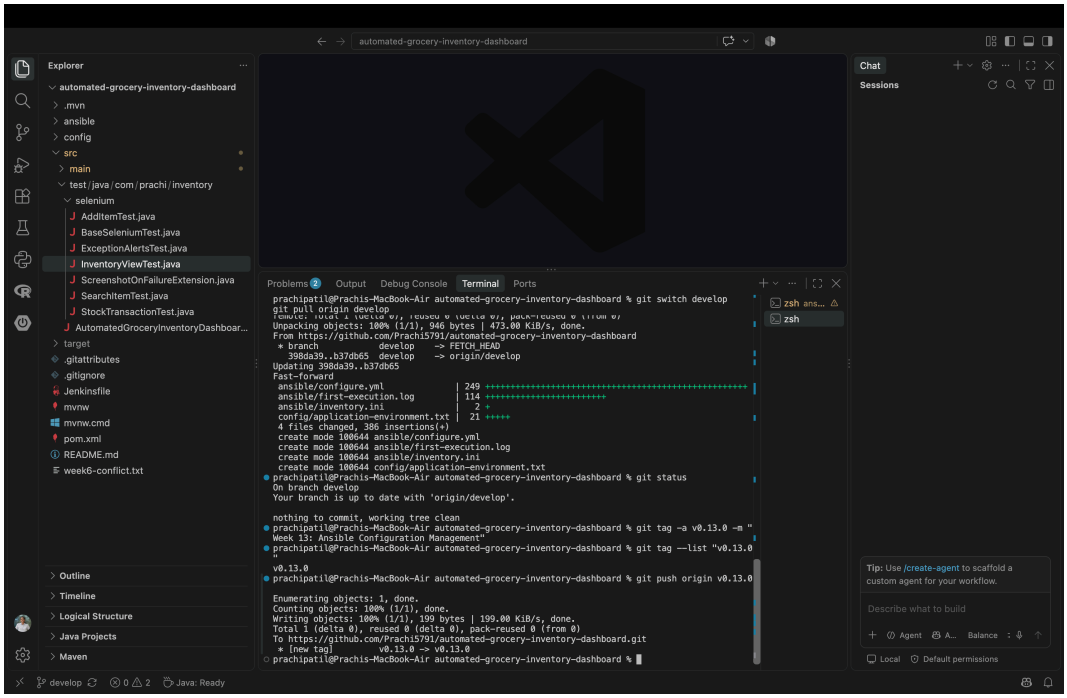


Figure 2: Week 13 baseline tag `v0.13.0`

8. 8. Final Validation

The Week 13 configuration-management implementation satisfies the required configuration areas:

Requirement	**Evidence**
Packages	Java, Maven, Git, Docker, MySQL and Ansible verified
User	Existing application user verified
Folders	Ansible and configuration directories managed
Files	Environment configuration file managed
Ports	3306, 8080, 8083 and 5001 verified
Services	Docker, application container and registry verified
Execution	23 tasks, 0 failures
Idempotence	Second run produced 0 changes
Version control	PR merged into `develop`
Baseline	`v0.13.0` pushed to GitHub

Table 3: Week 13 final validation

Conclusion

Week 13 introduced Ansible-based configuration management for the Grocery Inventory Dashboard. The target node was specified through an Ansible inventory, while the playbook established the required project directories and configuration file and verified the software, ports and services required by the application.

The successful second execution with zero changes demonstrates idempotent configuration management. The implementation was committed to Git, merged into the `develop` branch through a pull request, and preserved as the `v0.13.0` project baseline.

Week 13: Configuration Management – COMPLETE